Chapter 19. Gateway API

The Gateway API was introduced to standardize and build upon the lessons learned from Ingress and service mesh frameworks like Istio, Contour, and Linkerd, which had demonstrated the need for more sophisticated traffic management capabilities beyond what Ingress could provide. As a more expressive and extensible alternative to the traditional Ingress resource, the Gateway API offers role-oriented design, support for multiple protocols beyond HTTP/HTTPS, and enhanced traffic-routing features.

The Gateway API is the successor of the Ingress primitive and is becoming increasingly important as organizations adopt this modern approach to managing external traffic.

As a candidate to the exam, you will need to understand how to deploy the Gateway API and instantiate the relevant objects to stand up Ingress access to your cluster.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objective:

- Use the Gateway API to manage Ingress traffic

Why Is the Ingress Primitive Not Sufficient?

The Ingress API is the standard Kubernetes way to configure external HTTP/HTTPS load balancing for Services, but it has limitations. While the Ingress supports TLS termination and simple content-based request routing of HTTP traffic, real-world use cases call for more advanced features. Enhancing the existing Ingress API model of the Ingress wouldn't allow for adding those features easily for a variety of reasons:

Ingress controller-specific extensibility

Advanced features like traffic splitting, rate limiting, and request/response manipulation are provided by nonportable anno-

tations for specific Ingress implementations.

Insufficient permission model

The Ingress API is not well suited for multitenant environments that call for a strong permission model.

In this chapter, I am not going to discuss all possible configuration options and features of the Gateway API. That would go beyond the necessary coverage for the exam. However, I will explain how you can model ingress HTTP traffic similar to what you learned in Chapter 18. See the Gateway API documentation that explains how to implement other use cases.

Working with the Gateway API

The Gateway API is the official successor to the Ingress primitive, representing a superset of Ingress functionality, while at the same time enabling more advanced use cases.

You can think of the Gateway API as a unified and standardized API for managing traffic into and out of a Kubernetes cluster instead of having to choose from individual Ingress implementations. Product-specific annotations are not needed anymore to configure routing options. The Gateway API offers a flexible way to incorporate similar features. Effectively, the Gateway API is a universal specification supported by a wide range of different implementations.

Instead of handling one single primitive like the Ingress, the Gateway API splits up responsibilities into multiple primitives explained in the next section.

Gateway API Resources

The Gateway API introduces a layered approach to traffic management through four primary resource types that work together to handle incoming traffic:

Gateway

Defines an instance of traffic-handling infrastructure, such as a cloud load balancer.

GatewayClass

Each Gateway is associated with a GatewayClass, which describes the actual kind of gateway controller that will handle traffic for the Gateway.

HTTPRoute/GRPCRoute

Defines HTTP- or GRPC-specific rules for mapping traffic from a Gateway listener to a representation of backend network endpoints. These endpoints are often represented as a Service.

ReferenceGrant

Can be used to enable cross-namespace references within the Gateway API, e.g., routes may forward traffic to backends in other namespaces.

Managing those Gateway API resources falls within the responsibility of different personas, as shown in Figure 19-1.

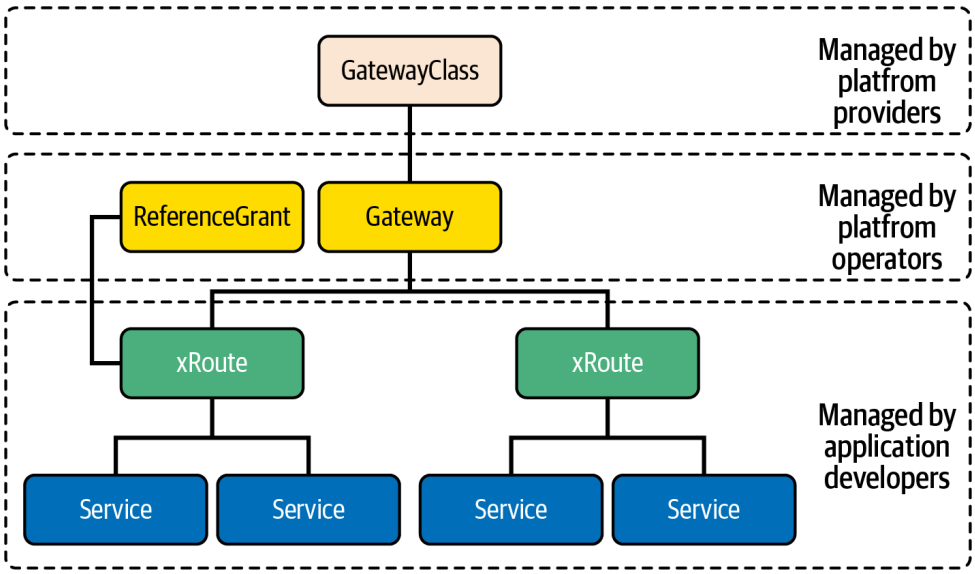

Figure 19-1. Gateway API resources managed by personas

The GatewayClass is provided by platform providers, e.g., the cloud provider. The Gateway and ReferenceGrant are installed by the Kubernetes cluster administrator. Lastly, the HTTPRoute and GRPCRoute are created by application developers.

The prominent Gateway API use case I want to demonstrate in this chapter is how to route HTTP ingress traffic to multiple Services based on a context URL. Figure 19-2 illustrates this use case.

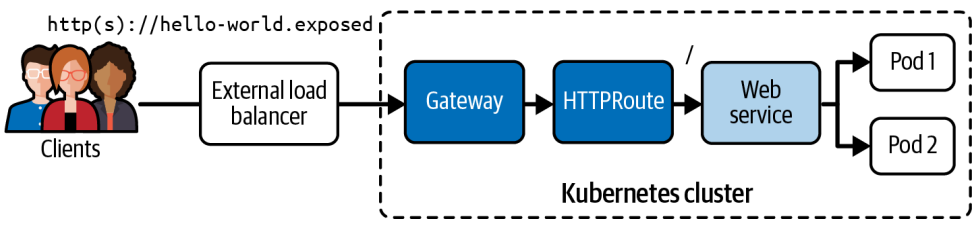

Figure 19-2. Gateway API HTTP traffic routing

Before we can actually create the objects shown in the visualization, we'll need to tell the Kubernetes cluster about the Gateway API CRDs.

Installing the Gateway API CRDs

At the time of writing, the Gateway API resources do not ship with the standard set of Kubernetes API resources. You will have to install the Gateway API in the form of Custom Resource Definitions. The following command installs version 1.3.0 of the CRDs:

```
$ kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/
download/v1.3.0/standard-install.yaml
customresourcedefinition.apiextensions.k8s.io/\
gatewayclasses.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/\
gateways.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/\
grpcroutes.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/\
httproutes.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/\
referencegrants.gateway.networking.k8s.io created
```

You will now be able to list the CRDs by searching for the API group used by the Gateway API resources:

```
$ kubectl get crds | grep gateway.networking.k8s.io
gatewayclasses.gateway.networking.k8s.io    2025-08-07T18:14:16Z
gateways.gateway.networking.k8s.io          2025-08-07T18:14:16Z
grpcroutes.gateway.networking.k8s.io        2025-08-07T18:14:16Z
httproutes.gateway.networking.k8s.io        2025-08-07T18:14:16Z
referencegrants.gateway.networking.k8s.io   2025-08-07T18:14:17Z
```

See the official GitHub repository for more information on the Gateway API.

Deploying a Gateway Controller

The Gateway API requires a controller implementation to function. Different controllers offer various features and performance characteristics. For this example, we'll use the Envoy Gateway implementation installed by Helm:

```
$ helm install eg oci://docker.io/envoyproxy/gateway-helm --version v1.4.2
  -n envoy-gateway-system --create-namespace
```

You will want to wait until the Gateway implementation becomes fully functional:

```
$ kubectl wait --timeout=5m -n envoy-gateway-system deployment/envoy-gatew
  --for=condition=Available
deployment.apps/envoy-gateway condition met
```

With the prerequisites in place, we'll set up everything needed to use the Gateway API to route Ingress HTTP traffic to a Service backend.

Creating GatewayClasses

Depending on the Kubernetes environment you are operating in, you may or may not have to create a GatewayClass. Cloud provider Kubernetes clusters should already come with one.

In case you want to create your own GatewayClass, you will first have to determine the Gateway controller name. The controller name depends on the controller implementation installed. You can usually look up the controller name in its documentation.

The controller name for Envoy is `gateway.envoyproxy.io/gatewayclass-controller`. Create the file *gateway-class.yaml* with the contents shown in Example 19-1.

Example 19-1. A GatewayClass that uses the Envoy GatewayClass controller

```
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: envoy
spec:
  controllerName: gateway.envoyproxy.io/gatewayclass-controller    ❶
```

❶ The reference to the controller installed by the Helm chart

Run the following command to create the GatewayClass object:

```
$ kubectl apply -f gateway-class.yaml
gatewayclass.gateway.networking.k8s.io/envoy created
```

To list GatewayClass objects, run the following command:

```
$ kubectl get gatewayclasses
NAME    CONTROLLER                                      ACCEPTED   AGE
envoy   gateway.envoyproxy.io/gatewayclass-controller   True       31s
```

You should see the object we created earlier, and potentially other GatewayClass objects that came with the Kubernetes environment.

Creating Gateways

With the GatewayClass instantiated, create a Gateway resource to handle incoming traffic. Create a YAML manifest file named *gateway.yaml* and populate the content, as shown in Example 19-2.

Example 19-2. A Gateway exposing an HTTP listener

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: hello-world-gateway
spec:
  gatewayClassName: envoy    ❶
  listeners:
    - name: http
      protocol: HTTP         ❷
      port: 80               ❷
```

❶ References the GatewayClass by name

❷ The listener accepting HTTP traffic on port 80

Run the following command to create the Gateway object:

```
$ kubectl apply -f gateway.yaml
gateway.gateway.networking.k8s.io/hello-world-gateway created
```

You list the Gateway object with the following command:

```
$ kubectl get gateways
NAME                  CLASS   ADDRESS   PROGRAMMED   AGE
hello-world-gateway   envoy             False        16s
```

There's only one additional object to set up to finalize the HTTP traffic routing: the HTTPRoute object.

Creating HTTPRoutes

Store the definition of the HTTPRoute in the YAML manifest file named *httproute.yaml*, shown in Example 19-3. For brevity, this chapter will not demonstrate how to create Service backends. Please reference Chapter 17 for more information.

Example 19-3. An HTTPRoute routing traffic to a Service backend

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: hello-world-httproute
spec:
  parentRefs:
    - name: hello-world-gateway
  hostnames:
    - "hello-world.exposed"      ❶
  rules:
    - backendRefs:
        - group: ""              ❷
          kind: Service          ❷
          name: web              ❷
          port: 3000             ❷
          weight: 1              ❸
      matches:
        - path:
            type: PathPrefix     ❹
            value: /             ❹
```

❶ The hostname to make traffic routing available for

❷ The Service backend to route the traffic to

❸ Determines the proportion of traffic that will be sent to that specific Service

❹ The routing rules based on the requested URL context

Run the following command to create the HTTPRoute object:

```
$ kubectl apply -f httproute.yaml
httproute.gateway.networking.k8s.io/hello-world-httproute created
```

You can list all HTTPRoute objects with the command shown:

```
$ kubectl get httproutes
NAME                    HOSTNAMES                  AGE
hello-world-httproute   ["hello-world.exposed"]    64s
```

Accessing the Gateway

Accessing the Gateway differs depending on the Kubernetes cluster you are using. Follow the instructions in the section based on your

Kubernetes cluster setup, and assume that you don't have external load balancer support.

Get the name of the Envoy Service created by the example Gateway, and assign it to the environment variable `ENVOY_SERVICE`:

```
$ export ENVOY_SERVICE=$(kubectl get svc -n envoy-gateway-system \
--selector=gateway.envoyproxy.io/owning-gateway-namespace=default,\
gateway.envoyproxy.io/owning-gateway-name=hello-world-gateway \
-o jsonpath='{.items[0].metadata.name}')
```

Port forward to the Envoy Service:

```
$ kubectl -n envoy-gateway-system port-forward service/${ENVOY_SERVICE} 88
[2] 93490
Forwarding from 127.0.0.1:8889 -> 10080
Forwarding from [::1]:8889 -> 10080
```

With the relevant entry in `/etc/hosts`, you should be able to make a call to the Gateway:

```
$ curl hello-world.exposed:8889
Handling connection for 8889
Hello World
```

In this example, the backend returns with the message "Hello World." A successful response may look different depending on the implementation details of your application.

Ingress to Gateway API Migration

The Gateway API represents the next evolution of traffic management in Kubernetes, designed to eventually replace the traditional Ingress resource.

The transition typically involves replacing Ingress resources with Gateway and HTTPRoute combinations. A single Gateway resource can handle multiple applications through different Route resources, similar to how an Ingress Controller works but with cleaner separation of responsibilities.

The Gateway API graduated to stable status for core features in late 2023. While Ingress remains supported and widely used, major ingress controllers like NGINX, Istio, and Envoy Gateway now offer Gateway API im-

plementations. Organizations should evaluate migration based on their need for advanced traffic management features and operational complexity requirements.

Most migrations follow a phased approach: deploying Gateway API alongside existing Ingress, gradually migrating applications, and eventually deprecating Ingress resources. Many controllers support both APIs simultaneously, allowing for smooth transitions without service disruption.

The official conversion tool named ingress2gateway automatically translates Ingress resources to Gateway API equivalents. It handles basic conversions and provides a foundation for more complex migrations. I am only mentioning this tool for completeness' sake. You are not expected to understand or use it in the exam.

Summary

The Gateway API represents the future of Kubernetes ingress management, offering powerful features while maintaining clarity through its role-oriented design. As more organizations adopt this standard, proficiency with the Gateway API becomes an essential skill for Kubernetes administrators.

Exam Essentials

Check on existing GatewayClasses

Remember that the exam environment may have preinstalled Gateway controllers, so always check available GatewayClasses before creating resources.

Understand the Gateway API CRDs

Practice creating different Gateway configurations, experiment with various HTTPRoute patterns, and understand how the components work together to build a solid foundation for managing production traffic.

Sample Exercises

Solutions to these exercises are available in Appendix A.

1. You are tasked with setting up ingress traffic management for a new application using the Gateway API. The application consists of two ser-

vices that need to be accessible from outside the cluster. Use the NGINX Gateway Fabric controller implementation.

Create a Deployment named `web-app` with two replicas using the `nginx:1.21` image in the `default` namespace. Create a Deployment named `api-app` with two replicas using the `httpd:2.4` image in the default namespace. Expose both Deployments as `ClusterIP` Services on port 80. You can shortcut the setup by applying the YAML file *setup.yaml* in the directory *app-a/ch19/basic-gateway* of the checked-out GitHub repository *bmuschko/cka-study-guide*.

Create a Gateway named `main-gateway` in the `default` namespace. Configure it to listen on port 80 for HTTP traffic. Use the hostname `example.local`. Create an HTTPRoute named `app-routes` that routes traffic with path `/web` to the `web-app` Service. It also routes traffic with path `/api` to the `api-app` Service. Define path prefix matching.

Check that the Gateway is ready and has an assigned IP/hostname. Verify that the HTTPRoute is accepted and bound to the Gateway, then test connectivity to the Gateway.

2. Your organization needs to set up a production-ready Gateway configuration that handles HTTP traffic for applications across multiple namespaces. Use the NGINX Gateway Fabric controller implementation.

   Create two namespaces named `production` and `staging`. In the `production` namespace, create a Deployment named `prod-web` with three replicas using the `nginx:1.22` image. Create a ClusterIP Service exposing port 80, which routes traffic to the replicas of `prod-web`. In the `staging` namespace, create a Deployment named `staging-web` with two replicas using the `nginx:1.21` image. Create a `ClusterIP` Service exposing port 80, which routes traffic to the replicas of `staging-web`. You can shortcut the setup by applying the YAML file *setup.yaml* in the directory *app-a/ch19/cross-namespace-gateway* of the checked-out GitHub repository *bmuschko/cka-study-guide*.

   Create a Gateway named `gateway` in the `production` namespace. Configure an HTTP listener on port 80. Use the hostname `example.com`.

   Create an HTTPRoute named `prod-route` in the `production` namespace. The object should route traffic with path `/app` to the `prod-web` Service, use exact path matching, and attach to the `gateway`. Create another HTTPRoute named `staging-route` in the `staging` namespace. The object should route traffic with path `/staging` to the `staging-web` Service, use path prefix matching, and attach to the `gateway` in the `production` namespace. Configure proper cross-namespace reference policy. Add request headers to identify the environment (production versus staging).

Verify the correct HTTP routing. The production route should only respond to the exact path `/app`. The staging route should respond to `/staging`. Both routes should successfully serve the NGINX welcome page.